Individual Reflection

CSC 481/581 - Game Engine Foundations - Fall 2026

Group 4: Sai Sumedh Kaveti

Milestone 3 - Runtime Game Object Models
Project 3 - Individual Reflection

Individual Game Name: "Cave Ninja" GitHub Game Repository:
https://github.com/CodeRockerr/CSC481-581-Group4/tree/main/Games/skaveti

(A) My Role

I owned the networking on the new object model and the CSC 581 performance comparison; Adit built the object model and Camera and Yeva built the gameplay systems. My work therefore depended on theirs, and I did it last. I replaced our fixed 29-byte message with a protocol which can send any game object, moved the server and client and peer-to-peer code onto the GameObjectManager, added a second data format, and ran the experiments the team uses. I then moved Cave Ninja onto the new model (Figures 4-7).

(B) Engine Design: Why a Component-Based Model

We chose a component style: a Game Object is a stable id plus a set of components, and the 'type' of an object is whatever components it contains. Our previous Entity had position, velocity, colour, gravity and texture on every object, so a hidden death zone carried a texture to never draw, and every feature was a change to the core class. Cave Ninja's paddedHitbox() helper was there because Entity used one rectangle for drawing and collision; collider padding replaced it.

An inheritance tree was not allowed and a platform that kills would have to have two parents. Property style is more cache-friendly and is easy to do as bulk serialization, but our scenes only have tens to a few hundred objects and our game engine asks one-object questions ('did the ninja touch the wisp?'). A full ECS is more engine than our 10-500 objects need.

The real tell was my own implementation: networking only needed one new component (Network Identity) and no change to Game Object. The wire format itself is property centric because every snapshot copies the same collection of properties from all the objects into one flat array. So we use components for gameplay and a property view for networking, the Module 7 trade-off inside one game engine.

Its limitations: get<T>() is a hash lookup, components are separate heap allocations (not cache friendly), and an object may only have one component of a given type; two-object behaviour still lives in game code; thread safety depends on a rule: only the main thread may create or destroy objects.

(C) Network Architecture

Client-server is our primary model; I also moved Adit's peer-to-peer mode to the same packets. The server has ownership of the moving platforms and computes Yeva's MovementPath on a 60 Hz thread so that all clients see the same platforms. Each client owns and simulates its own ninja (no input lag) and the server relays it, at the expense of trusting clients not to cheat.

Identity: Objects are identified by (owner, netId), with owner 0 being the server. Our plan called for creating replicas with createWithId(netId) but ids collide across machines (the server's initial platform and every client's background are both id 1) and the call advances the local id counter. Therefore, each endpoint's NetReplicator maps (owner, netId)

to a local id and the server overwrites the owner field so that a client can only update its own objects.

Lifecycle: Each reply is a complete snapshot: an object appearing is a create, reappearing an update, and vanishing a destroy. A lost packet is simply replaced with the next snapshot.

Threads: The server has an accept thread, one thread per client on its own port (thus allowing an arbitrary number of clients) and the simulation thread; each client has a network thread. The game thread is never blocked: publish() buffers owned objects and sync() applies the newest snapshot on the main thread, so workers are never allowed to alter the object list.

Disconnects: A client that sends Leave, or is silent for 3 seconds (a crash or a closed window), is dropped; its objects vanish from the next snapshot and every other client destroys the replica (Figure 7).

Data format: One 0MQ frame of a 22-byte header and a 36-byte ObjectState per object (netId, owner, kind, flags, frame, position, size, velocity). The first byte specifies an encoding, so that one server serves both formats.

(D) Performance Analysis: Binary vs Text

Text was up to 7.5 times slower than binary, and this grew as a function of the number of objects, while the text packets were only about 1.5 times bigger. That means the cost of text is CPU time doing the formatting and parsing of numbers, not network bandwidth.

The two formats: Binary does memcpy(22 + 36n bytes); text does snprintf, then strtof (about 56 bytes and 11 numbers) per object. Both contain the same fields, so gameplay is the same; each client picks which it wants via a command-line switch.

Experiment	Case	Binary mean (ms)	Text mean (ms)	Text / binary	Std dev (ms) bin / text
E1 objects	10	363	421	1.2x	1.6 / 3.0
E1 objects	100	403	864	2.1x	1.3 / 4.3
E1 objects	500	609	3,173	5.2x	4.7 / 12.3
E2 clients	1	406	871	2.1x	9.8 / 17.6
E2 clients	2	497	956	1.9x	4.6 / 2.9
E2 clients	4	648	1,164	1.8x	1.3 / 44.1
E3 owned per client	1	479	529	1.1x	35.3 / 10.9
E3 owned per client	100	550	1,790	3.3x	1.6 / 15.4
E3 owned per client	500	931	6,962	7.5x	3.0 / 82.0

Method: A headless client runs 5,000 iterations, after 200 warm-up, publishing, waiting for a new snapshot to come in, then applying it (a round trip). E1 varied moving objects (10/100/500, 1 client), E2 varied number of clients (1/2/4, 100 objects), and E3 varied objects owned per client (1/100/500, 2 clients). Each case ran 5 times on a fresh server on my MacBook Air, over loopback, per format, and I report the mean and sample variance (Figures 1-3).

Elapsed time for 5000 iterations, the mean of 5 runs. Variances are in Docs/Results/summary.csv.

Analysis: At 500 objects text packets were 1.55x bigger but 5.2x slower, which suggests that CPU is the limiting factor, not the bandwidth. Binary also scales (363 to 609 ms), but it has a lower per-object overhead, about 10x smaller than text. E3 is the worst case for text due to objects being encoded twice, and each snapshot carrying around 1010 objects (500 from each client plus 10 server objects). As we add clients in E2, the ratio decreases to 1.8x due to contention being a bigger factor, impacting both the encoders. The 100-object E1 and 1-client E2 are essentially the same scenario and have approximately the same results, within 1% of each other, which shows that the benchmarks are repeatable.

Limits: One laptop is used in a loopback scenario, which removes real network latency, with both encoders running on the same cores, and one iteration is a round trip, not a rendered frame.

Conclusion: Binary encoding should be the default, and text should only be used for inspecting packets while debugging.

(E) Cave Ninja: Using the Model

Every object in Cave Ninja is now a GameObject built from components, and no object type is a class:

Object	Components
Ninja (mine)	Transform, Sprite (4 frames), Collider (padded), PhysicsBody (PlayerTime, gravity), PlayerInput, NetworkIdentity
Ninja (other players)	Transform, Sprite, NetworkIdentity marked as a replica; no physics, its position comes from its owner
4 static platforms	Transform, Sprite (stone, icy blue, ember red, moss green; all different sizes), Collider, Ground
3 moving platforms	Transform, Sprite, Collider, Ground, MovementPath (horizontal, vertical, circular); server-owned when networked
Shadow wisp	Transform, Sprite, trigger Collider, Patrol
3 spawn points	Transform, SpawnPoint (start, mid, late); hidden because they have no Sprite
2 death zones	Transform, trigger Collider, DeathZone; hidden
2 scroll boundaries	Transform, trigger Collider, ScrollBoundary; hidden, pinned to the view

The only game-specific components are Ground, Patrol, and ScrollBoundary. Whatever kills him, the ninja returns to his last checkpoint: I set every DeathZone's spawn point to the current checkpoint. Two hidden boundaries at 30% and 60% of the view move the camera by however far the ninja passes them (Figure 6). Pressing H draws all hidden objects (Figure 5). With a server and three clients, every window shows the same scene (Figure 4).

(F) What Did Not Work at First

The ninja fell through the world. After a stall (resize or drag), one big time step carried him past the 200-unit death zone, so it did not trigger. I made the zones extend much further below the world and added a check for below the world; the proper fix would be to bound the time-step in the shared engine loop.

The camera was not resetting. Yeva's SideScrollTrigger only moves the camera forward, once. My first implementation of a boundary only triggered a scroll if the player overlapped it, which a teleport does not. I now check positions and re-centre the camera every time the player respawns.

Respawning rules were unclear. Pits and the wisp had different respawn rules, and the checkpoint did not reset, so dying to the wisp at the start sent the player forward to a later checkpoint. This was found in playtesting; I changed all respawn rules to the same one.

createWithId was not working correctly across the network. It would crash on the first replicated object, because ids are unique per-machine, so the (owner, netId) solution was used instead.

As in Milestone 2, rules that depend on a fixed time-step will not work across the network, due to varying real time between frames and teleports. Merging last taught me to make small, frequent commits and then I had to rebase my branch onto Yeva's after she pushed.

(G) Limitations and What It Enables

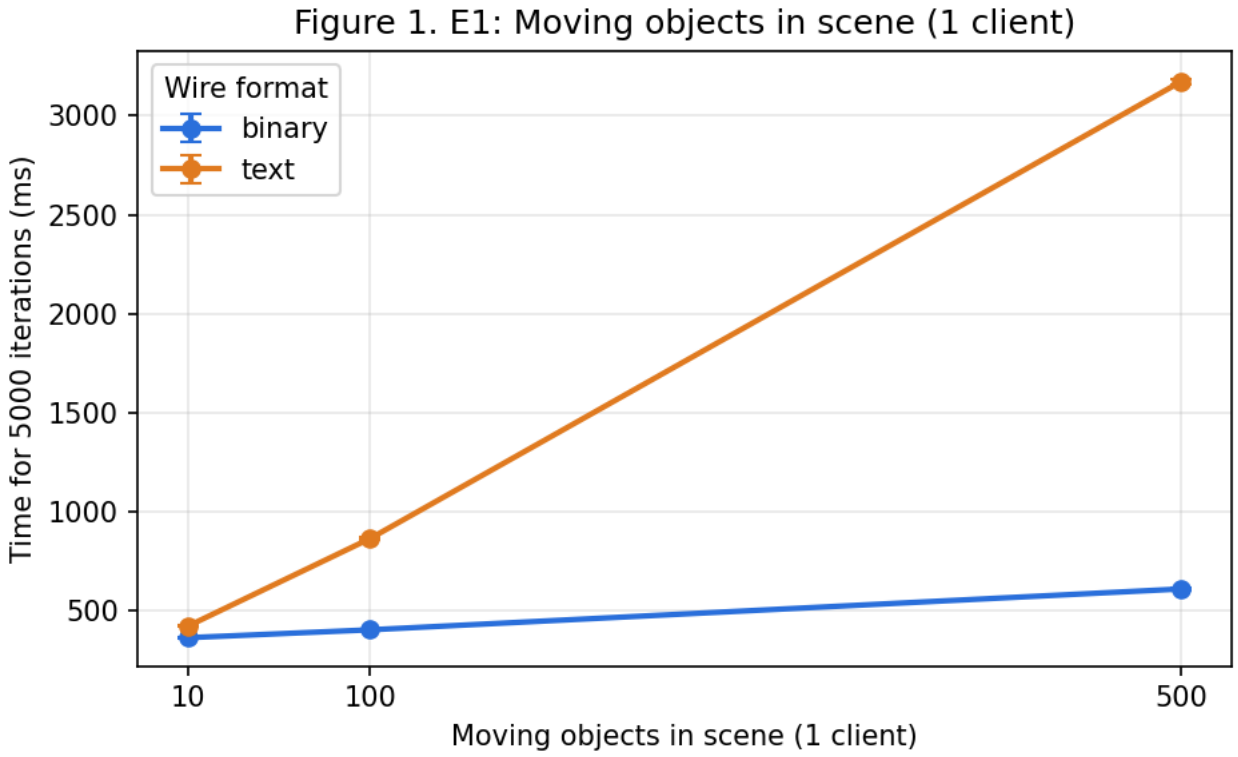
Limitations: Every snapshot contains every object, including those that are not moving. Players are client-authoritative, meaning that a modified client could cheat by overriding their movements on the client side. Remote ninjas are not interpolated. And the demo uses binary data, which assumes that the client and server are using the same byte order.

What it enables: A new networked object type can be added with a minimum of fuss: just add a new kind and a spawn callback, without any protocol changes. Server-side checks, such as rejecting impossible speeds, are possible since the server has access to everything. And a third option, such as sending only changed values, can be experimented with using the existing code.

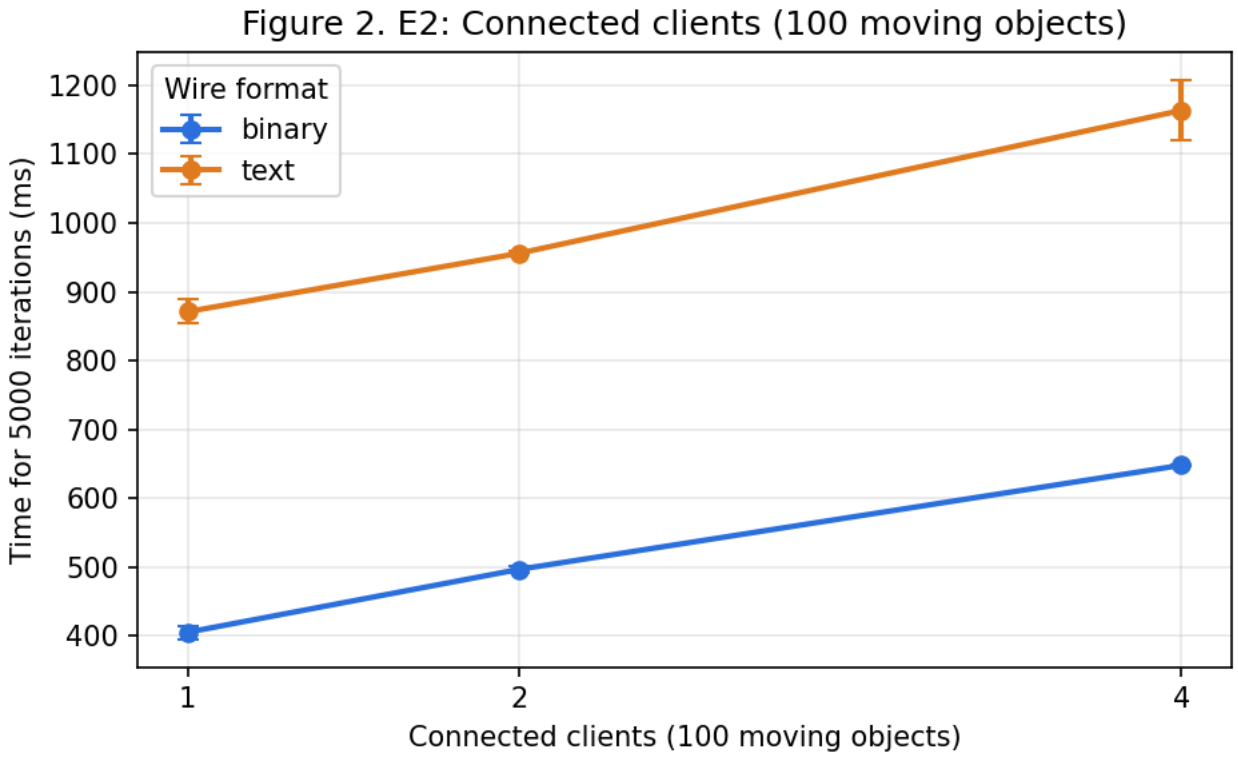
(H) Attribution

The object model comparison is inspired by the Module 7 lecture about the runtime game object model. The object model and Camera belong to Adit, the peer-to-peer design that I did the migration from is his Milestone 2 code, and MovementPath, SpawnPoint and DeathZone are Yeva's gameplay systems. The networking was implemented using libzmq 4.3.5 and rendering using SDL3 with stb_image binding for loading textures.

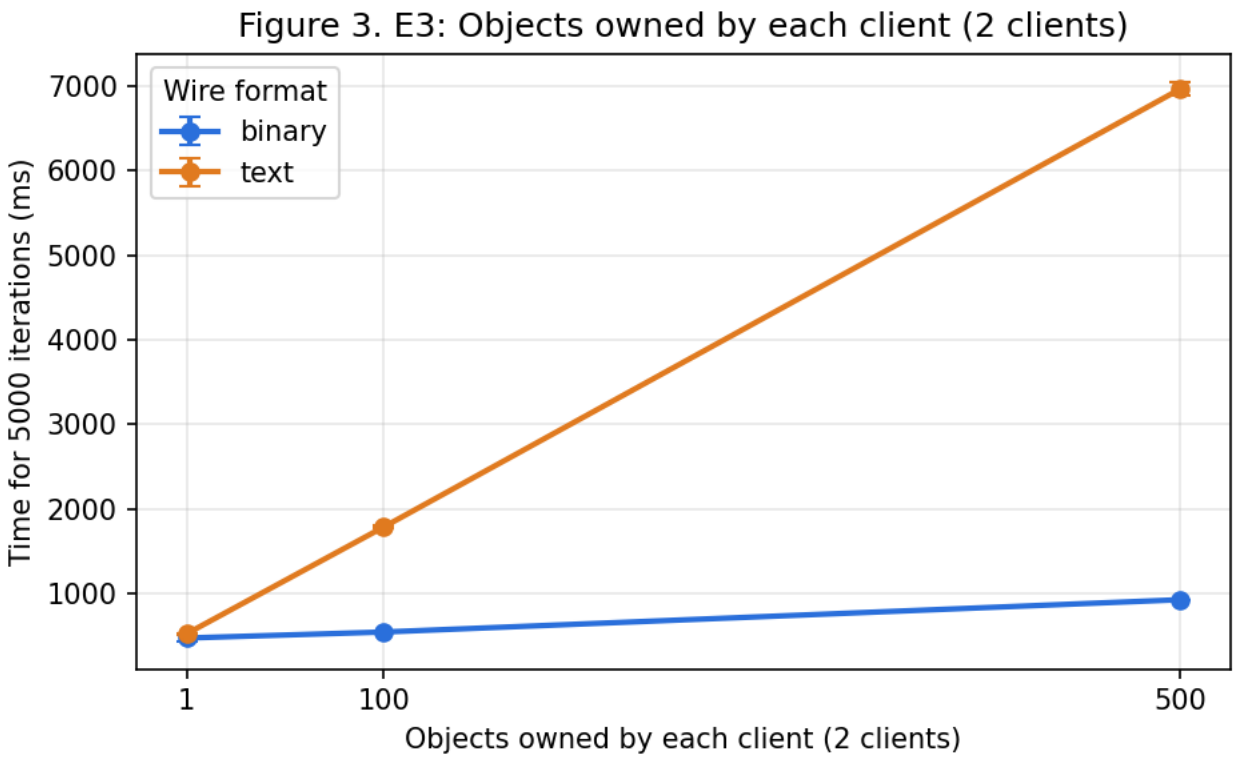
Appendix - Screenshots



(fig. 1 - E1: time for 5,000 iterations as the scene grows from 10 to 500 moving objects, 1 client)



(fig. 2 - E2: time for 5,000 iterations with 1, 2 and 4 clients, 100 moving objects)



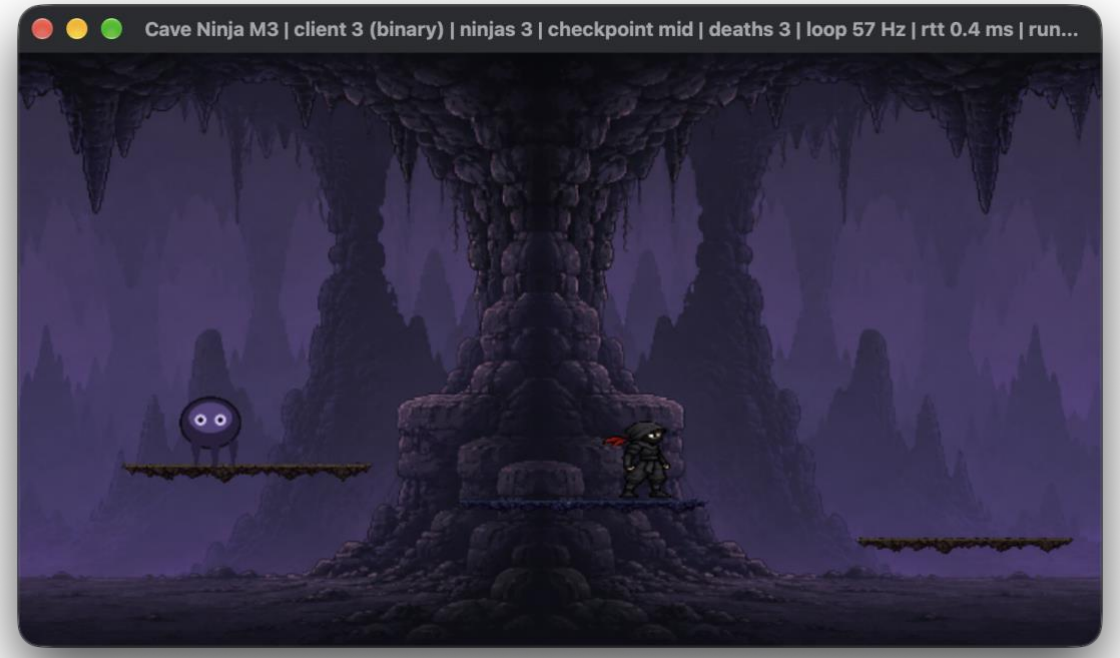
(fig. 3 - E3: time for 5,000 iterations as each of 2 clients owns 1, 100 and 500 objects)



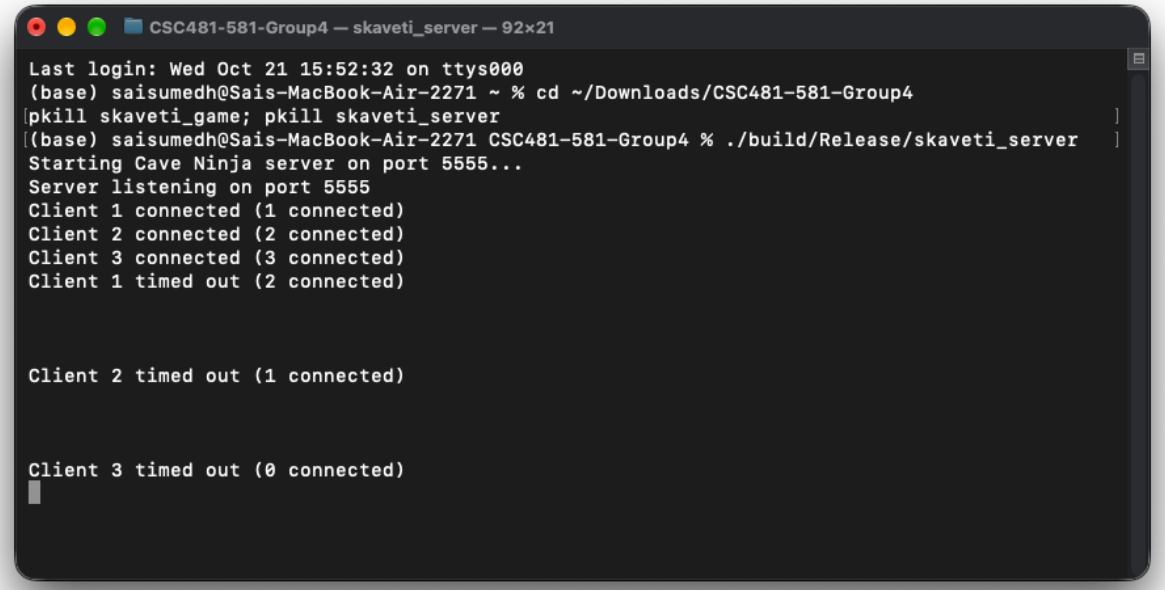
(fig. 4 - Server + 3 clients (binary, text, binary): same platforms and all three ninjas in every window)



(fig. 5 - Debug view (H): hidden spawn points in green, death zones in red, scroll boundaries in blue)



(fig. 6 - Side scrolling: the camera has followed the ninja past the first screen to the mid checkpoint; the wisp's moving platform is on the left)



(fig. 7 - Server terminal: three clients connect; each disconnected client is detected (here by the 3-second timeout) and removed, and the remaining clients keep running)